Products Caching Refactor — Steps 4 to 6

Requirement 3 (Performance & Caching) · Branch `refactor/products-caching-decorator` · 12 November 2026

Where we are. Steps 0–3 moved Products onto a repository (`IProductRepository` / `ProductRepository`), removed the old `ProductCache` that was mixed into `ProductService` , and added a per-request `CacheStatus` plus the middleware that writes the `X-Cache` header. This document covers the last three steps:

- **Step 4** — caching re-added as a *decorator* around the repository, with a Redis failure switch and config-driven wiring.
- **Step 5** — limits on the product-list query (page, page size, category).
- **Step 6** — documentation updated, and the whole thing checked end to end against the running system.

The final shape

```
HTTP GET /api/v1/Products/{id}
  │
  ├─ CacheStatusHeaderMiddleware ........ writes X-Cache just before the response is sent
  │
  ├─ ProductsController  [ETag] ......... 304 when the body hash matches If-None-Match
  │
  ├─ IProductService
  │    └─ LoggingProductServiceDecorator
  │         └─ ProductService ............ business rules
  │              ├─ Inventory gRPC client  live stock, never cached
  │              └─ IProductRepository
  │                   └─ CachingProductRepositoryDecorator ── Redis
  │                        │   records HIT / MISS / BYPASS in CacheStatus
  │                        └─ ProductRepository ── PostgreSQL
  │
```

Each concern sits in its own layer: **logging** and **caching** are decorators, **business rules** are in `ProductService` , **data access** is in `ProductRepository` , and **HTTP details** (headers, ETag) stay in the HTTP pipeline. That matches the decorator convention the rest of the codebase already uses.

Step 4 — The caching decorator

Goal: bring caching back without touching `ProductService` or `ProductRepository` . The decorator implements the same interface as the repository, so the service cannot tell whether it is talking to Redis or Postgres.

4.1 Settings class — `Products/Caching/ProductCacheOptions.cs`

```
public sealed class ProductCacheOptions
{
    public const string SectionName = "Caching";
    public bool Enabled { get; set; } = true;
    public int ProductTtlSeconds { get; set; } = 300;     // single product: 5 minutes
    public int ProductListTtlSeconds { get; set; } = 60;  // list pages: 1 minute
}
```

- This is .NET's **options pattern**: the `Caching` section of `appsettings.json` is loaded once into a typed class, instead of reading `config["Caching:..."]` strings around the code.
- Property names match the JSON keys; the defaults apply if a key is missing.
- **Why lists expire sooner than single products:** a page goes stale for more reasons (any product on it changes, or a new product is added), so it gets a shorter safety net.

4.2 Redis failure switch — `Shared/Caching/CacheCircuitBreaker.cs`

```
public sealed class CacheCircuitBreaker
{
    private readonly TimeSpan _cooldown;
    private long _skipUntilTicks; // UTC ticks; 0 means "Redis is fine"

    public CacheCircuitBreaker(TimeSpan cooldown) { _cooldown = cooldown; }

    public bool IsOpen => DateTime.UtcNow.Ticks < Interlocked.Read(ref _skipUntilTicks);

    public void ReportFailure() =>
        Interlocked.Exchange(ref _skipUntilTicks, DateTime.UtcNow.Add(_cooldown).Ticks);
}
```

> **Problem it solves:** when Redis is down, every Redis call waits for the 500 ms timeout before failing. A list request makes 2–3 Redis calls, so each request could get ~1.5 s slower. The breaker remembers "Redis failed — skip it until *now + 30 s*", so only the first request pays the timeout.

- **"Open" means the circuit is broken** — calls don't go through to Redis. After the cooldown, the next request tries Redis again; if it works, caching resumes on its own.
- **Singleton, not per request:** the "Redis is down" knowledge must be shared, otherwise every request would rediscover it by waiting for a timeout.
- **`Interlocked`:** many requests on different threads read and write `_skipUntilTicks` at the same time. `Interlocked.Read` / `Exchange` make each read and write atomic, so no thread ever sees a half-written `long`.
- **In `Shared`** because it protects the Redis connection itself, not something specific to Products.

4.3 The decorator — `Products/Decorators/CachingProductRepositoryDecorator.cs`

It implements `IProductRepository` and holds another `IProductRepository` (`_inner`, the EF repository). Same interface outside, extra behaviour inside, real work delegated — the same shape as `LoggingProductServiceDecorator`.

Reads — the cache-aside pattern

```
public async Task<Product?> GetByIdAsync(Guid id, CancellationToken ct = default)
{
    var key = ProductKey(id);                                  // "product:{id}"

    var cached = await TryGetAsync<Product>(key, ct);          // 1. look in Redis
    if (cached is not null) return cached;                     // 2. HIT: database never touched

    var product = await _inner.GetByIdAsync(id, ct);           // 3. MISS: ask the database
    if (product is not null)                                   // 4. store for next time (not-found is not
cached)
        await TrySetAsync(key, product, TimeSpan.FromSeconds(_options.ProductTtlSeconds), ct);

    return product;
}
```

`GetPageAsync` follows the same four steps for list pages, with a 60-second expiry.

Writes — commit first, then invalidate

```
public async Task UpdateAsync(Product product, CancellationToken ct = default)
{
    await _inner.UpdateAsync(product, ct);   // the database is committed first...
    await InvalidateAsync(product.Id);       // ...then the stale cache entries are dropped
}
// AddAsync and RemoveAsync are identical in shape.
```

> **Why this order:** if you invalidated first, another request could reload the *old* row from the database and put it back in Redis before your commit lands — leaving stale data cached for the full TTL.

The list "version" trick

```
products:list:{version}:{category}:{page}:{pageSize}
e.g. products:list:3f9a1c…:electronics:1:20
```

- Every list key contains a version token stored under `products:list:version`.
- On any write, the version gets a new random value. From then on every request builds keys with the new version, so old pages are simply never read again and expire after 60 s.
- This beats searching Redis for every `products:list:*` key, which is slow and blocks Redis while it runs.
- If the version can't be read (Redis down), there is no safe key, so `TryGetPageKeyAsync` returns `null` and that request skips the cache.

Invalidation is deliberately stricter than reads

```
// Always attempted, even when the breaker is open, and never cancelled:
// the database has already changed, so skipping this could leave stale data in Redis.
private async Task InvalidateAsync(Guid productId)
{
    try
    {
        await _cache.RemoveAsync(ProductKey(productId));
        await _cache.SetStringAsync(ListVersionKey, Guid.NewGuid().ToString("N"), …);
    }
    catch (Exception ex)
    {
        _breaker.ReportFailure();
        _logger.LogWarning(ex, "Cache invalidation failed …; stale entries will expire by TTL", productId);
    }
}
```

- **Ignores the breaker:** writes are rare, so an occasional timeout is acceptable. Skipping while Redis had actually recovered would leave stale data cached.
- **No request `CancellationToken`:** if the client disconnects right after the commit, the invalidation must still run.

Error handling and the `X-Cache` header

- `catch (Exception ex) when (ex is not OperationCanceledException)` catches Redis errors but lets cancellation through — a cancelled request is not a Redis failure and must not open the breaker.
- On a Redis error: record `BYPASS`, open the breaker, log one warning, read from the database. The cache is never a single point of failure.
- `_status.Record(...)` writes HIT / MISS / BYPASS into the per-request `CacheStatus` from step 3, which the middleware turns into the `X-Cache` header.
- `JsonSerializerDefaults.Web` (camelCase) is used for the stored JSON; `Product` and `ProductPage` serialize as they are, so no separate "cached copy" types are needed.

4.4 Wiring — `Products/ProductsModuleExtensions.cs`

```
public static IServiceCollection AddProductsModule(this IServiceCollection services,
                                                    IConfiguration configuration, string connectionString)
{
    …
    var cacheSection = configuration.GetSection(ProductCacheOptions.SectionName);
    services.Configure<ProductCacheOptions>(cacheSection);                      // makes IOptions<…>
injectable
    var cacheOptions = cacheSection.Get<ProductCacheOptions>() ?? new();        // read Enabled at startup

    services.AddScoped<ProductRepository>();                                    // the real repository

    if (cacheOptions.Enabled)
        services.AddScoped<IProductRepository>(provider => new CachingProductRepositoryDecorator(
            provider.GetRequiredService<ProductRepository>(),
            provider.GetRequiredService<IDistributedCache>(),
            provider.GetRequiredService<CacheStatus>(),
            provider.GetRequiredService<CacheCircuitBreaker>(),
            provider.GetRequiredService<IOptions<ProductCacheOptions>>(),
            provider.GetRequiredService<ILogger<CachingProductRepositoryDecorator>>()));
    else
        services.AddScoped<IProductRepository>(provider => provider.GetRequiredService<ProductRepository>());
    …
}
```

- This is the project's existing pattern: concrete class registered as itself, interface resolved to a decorator around it (exactly how `IProductService` is wired).
- **Caching off = the decorator does not exist in the app.** No `if (_enabled)` checks anywhere and no Redis calls. This is the real fix for the old bug where the "before" benchmark still talked to Redis.
- `ProductService` still just asks for `IProductRepository` and never knows which one it gets.

4.5 `Host/Program.cs` and 4.6 `appsettings.json`

```
// Program.cs
var cacheCooldown = TimeSpan.FromSeconds(builder.Configuration.GetValue("Caching:FailureCooldownSeconds",
30));
builder.Services.AddSingleton(new CacheCircuitBreaker(cacheCooldown));   // one shared switch
…
builder.Services.AddProductsModule(builder.Configuration, connectionString);

// appsettings.json
"Caching": { "Enabled": true, "ProductTtlSeconds": 300, "ProductListTtlSeconds": 60,
"FailureCooldownSeconds": 30 }
```

The breaker is registered in `Program.cs` because Redis itself is registered there; the switch belongs to the Redis connection, not to one module.

Step 5 — Limits on the product list

Problem: `GET /api/v1/Products` and `POST /api/v1/Products/search` accepted any `page` and `pageSize`:

- `page=0` made EF skip a negative number of rows → an unhandled error (500).
- `pageSize=100000` would load 100,000 rows, fire 100,000 gRPC stock calls at once, and store one huge page in Redis.
- Every new `pageSize` or `category` value creates a new cache key, so unbounded input means unbounded keys.

Fix in `ProductsController` — one helper used by both endpoints, returning the project's standard `ProblemDetails` response through `BaseController.ProblemResponse`:

```
private const int MaxPageSize = 100;
private const int MaxCategoryLength = 100;

private IActionResult? ValidateListQuery(string? category, int page, int pageSize)
{
    if (page < 1)
        return ProblemResponse(400, "Invalid pagination", "page must be 1 or greater.");
    if (pageSize < 1 || pageSize > MaxPageSize)
        return ProblemResponse(400, "Invalid pagination", $"pageSize must be between 1 and {MaxPageSize}.");
    if (category is { Length: > MaxCategoryLength })
        return ProblemResponse(400, "Invalid category", $"category must be at most {MaxCategoryLength}
characters.");
    return null;
}

[HttpGet, ETag]
public async Task<IActionResult> GetProducts([FromQuery] string? category, [FromQuery] int page = 1,
[FromQuery] int pageSize = 10)
{
    var invalid = ValidateListQuery(category, page, pageSize);
    if (invalid is not null) return invalid;
    …
}
```

- The limits match Lisa's orders endpoint (page ≥ 1, page size 1–100), so the API behaves consistently.
- The category length limit was added because the category is part of the cache key.
- Validation sits in the controller, where request input is checked, not in the cache or the repository.

Step 6 — Documentation and end-to-end check

6.1 Documentation

- `docs/performance-testing.md` (Arabic): the file table now lists the repository, the decorator, the options class, the breaker and the `X-Cache` pieces; it adds the decorator chain diagram, the Redis-failure behaviour, the list limits, and the known stale-write limitation. The obsolete `ProductCache` references are gone.
- `CLAUDE.md`: the "Decorator pattern" convention now covers repositories and caching (cross-cutting concerns go in `Decorators/`, not inside services), and the caching section describes the new design.

6.2 End-to-end results

Run against the real system: Postgres and Redis in Docker, the Node inventory service, and the API on `http://localhost:5169`, using a local test admin account.

Check	Expected	Actual
Product list, twice	MISS, then HIT	**MISS → HIT ✓**
Single product, twice	MISS, then HIT	**MISS → HIT ✓**
Same product with its ETag in `If-None-Match`	304, from cache	**304 + X-Cache: HIT ✓**
Admin edits the price, then GET with the *old* ETag	200 with new price, MISS	**200, new price, new ETag, MISS ✓**
Same product again after the edit	HIT	**HIT ✓**
Product list after the edit	MISS (version changed)	**MISS ✓**
`page=0` / `pageSize=0` / `pageSize=101`	400	**400 / 400 / 400 ✓**
`pageSize=100`	200	**200 ✓**
Redis stopped, list requested 3 times	BYPASS; only the first is slow	**BYPASS × 3 — 0.77 s, then 0.015 s, 0.013 s ✓**
Redis started again, wait 30 s	Caching resumes on its own	**HIT, HIT ✓**
Redis failures logged	One warning, not one per request	**1 warning ✓**
API started with `Caching__Enabled=false`, Redis stopped	No X-Cache header, no Redis activity	**No header, no Redis log lines, ETag still works ✓**

The edited test product was restored to its exact original name, description, price, category and image afterwards. (During testing, the first edit briefly saved a shortened description because the test script split it on spaces; the restore was verified field-by-field.)

Known limitations and follow-ups

> **Stale-write race (accepted, documented).** If a request reads the old row from the database just before an edit commits and writes it to Redis just after the invalidation, the old data can stay cached until it expires (at most 5 minutes for a product, 1 minute for a list page). The window is very small; closing it fully needs per-product versioning or a delayed second delete, which wasn't worth the complexity here.

- **Stock is still one gRPC call per product** (20 per page, run in parallel). A batched `CheckQuantities(repeated ProductId)` RPC in `inventory.proto` would make it one call per page — the biggest remaining speed-up, and it also answers the N+1 part of the requirement.
- **ETag saves bandwidth, not server time:** the whole request still runs before the filter decides on 304. Measure response bytes as well as p50/p95/p99 in JMeter.
- **Cache stampede:** every product create/update changes the list version, so the next burst of list requests all go to the database at once. Fine at this scale.
- **Still open from Requirement 3:** large fake dataset with `EXPLAIN` analysis and indexes, and the read replica.
- The local test admin `cache-test-admin@example.test` created for this check is still in the local database.

Files changed in steps 4–6

File	Change
`src/Modules/Products/Caching/ProductCacheOptions.cs`	New — typed cache settings
`src/Shared/Caching/CacheCircuitBreaker.cs`	New — Redis failure switch
`src/Modules/Products/Decorators/CachingProductRepositoryDecorator.cs`	New — all caching logic
`src/Modules/Products/ProductsModuleExtensions.cs`	Takes `IConfiguration`; registers the decorator only when caching is on
`src/Host/Program.cs`	Registers the breaker; passes configuration to the Products module
`src/Host/appsettings.json`	Adds `Caching:FailureCooldownSeconds`
`src/Modules/Products/Controllers/ProductsController.cs`	Page / page size / category limits on both list endpoints
`src/Shared/Http/CacheStatusHeaderMiddleware.cs`	Moved out of the accidental `Http/src/Shared/Http/` folder (no code change)
`docs/performance-testing.md`, `CLAUDE.md`	Updated to the new design